Programming Problem-Solving Studio

C Programming | Classroom Team Activity | November 2–3, 2026

Team worksheet. Complete this on paper during the 40-minute breakout on November 2. It guides your discussion; **it is not a grading rubric**. Pseudocode, diagrams and flowcharts are sufficient. A complete C program is not required.

Group ID: _____ **Problem number:** _____ **Date:** _____

Record participating members in the table on page 4. Attach it to your worksheet.

A. Problem formulation

What will your program accomplish? State its inputs, expected outputs, scope and assumptions.

B. Data representation and design choices

What must the program remember? Choose suitable variables, arrays, strings, structures or other representations. Identify the main functions or operations and explain one important design choice.

Team worksheet (continued)

Group ID: _____ **Problem number:** _____

C. Algorithm, pseudocode or flowchart

Describe the sequence of operations, decisions, repetitions and function calls. Make the connection between inputs and outputs clear. Attach another sheet if needed.

D. Testing and verification

Give at least two tests, including one boundary or error case. State the expected result precisely enough to check your algorithm by hand.

Test	Input or situation	Expected result
Normal case		
Boundary or error case		

E. Learning and reflection

What was the main design decision? What difficulty or disagreement arose? What would you improve with more time?

Problem statements

Each problem permits several valid solutions. Choose a manageable scope and state your assumptions. Group and problem assignments are provided separately.

Problem 1: Railway ticketing

Design a program to book a railway ticket.

Problem 2: Library

Design a program to find and borrow a book from a library.

Problem 3: Banking

Design a program to process bank deposits and withdrawals.

Problem 4: Inventory management

Design a program to identify items that need restocking in a shop or warehouse.

Problem 5: Data center – a systems engineering problem

A data center contains servers that operate continuously. They consume electrical power, perform computations and generate heat. Their operation depends on several engineering considerations:

- **Electrical Engineering:** processor power consumption, operating frequency, temperature and computational performance.
- **Mechanical Engineering:** airflow, cooling fans, heat transfer and temperature regulation.
- **Metallurgical and Materials Engineering:** thermal conductivity, material properties and suitable materials for heat sinks and cooling components.

Sensors and monitoring systems collect measurements about the equipment. **Design a program that helps monitor or improve some aspect of a data center's operation.**

Choose a subsystem or question. Decide which measurements are relevant, how to represent them and what conclusions or decisions the program should support. Discuss how the electrical, cooling and material considerations relate to your chosen scope. Simple rules and stated assumptions are sufficient; advanced engineering equations and a model of the entire data center are not required.

Learning outcomes

By the end of the activity, you should be able to:

1. Turn an open-ended request into a scoped problem with explicit inputs, outputs and assumptions.
2. Select a data representation and divide a solution into functions or operations, explaining the choices.
3. Describe an algorithm clearly enough for another person to trace by hand.
4. Design normal and boundary/error tests with expected results, and use them to identify errors or missing assumptions.
5. Compare alternative designs and explain one limitation or improvement.

Programming concepts

Use variables, data types, input/output, conditionals, loops, arrays, functions, strings, structures, pointers, dynamic memory, searching, sorting or validation where they help. **You do not need to use every construct.** Explain why each chosen construct is appropriate. Do not add a C feature solely to include it.

Team record and session guide

Group ID: _____ **Problem number:** _____

Enter the details of each participating member. Leave unused rows blank.

No.	Full name	Full student ID	Branch
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

November 2: team breakout (40 minutes)

Use this sequence as a guide. Every member should contribute to the discussion.

Minutes	Task
0–7	Agree on scope, inputs, outputs and assumptions (A).
7–15	Choose data representations and main operations (B).
15–27	Develop and trace the pseudocode or flowchart (C).
27–35	Apply normal and boundary/error tests; revise the design (D).
35–40	Record learning, unresolved questions and improvements (E).

Team deliverable: one completed worksheet, plus any attached diagrams or pseudocode. Include the group ID, problem number and participating members' names and IDs. Provide all five sections and at least two tests with expected results. Keep the record available for the synthesis class. No executable program or slides are required.

November 3: synthesis and presentation protocol

The instructor will select teams or individuals to present; not every team will necessarily present. Each selected presentation has **3 minutes**, followed by **1 minute for a question or comparison**.

- State the problem scope and one assumption (about 30 seconds).
- Explain the representation and main algorithm (about 90 seconds).
- Trace one test and state one limitation or improvement (about 60 seconds).

Use the worksheet or board. Any member may be asked to explain a decision. Other teams should compare assumptions, representations or tests and ask specific questions. The instructor will synthesise common programming principles, including reusable operations, correctness and how designs change as data grows.